

Guia de Despliegue y Operaciones

El despliegue principal está preparado con **Docker Compose**. El archivo docker-compose.yml define tres servicios: db, backend y frontend. Cada servicio tiene su propio contenedor, puertos y variables de entorno.

Servicio	Imagen / build	Puerto host	Puerto contenedor	Función
db	mysql:8.0	3307	3306	Base de datos MySQL con volumen persistente y scripts SQL iniciales.
backend	./backend/Dockerfile	8000	8000	API FastAPI ejecutada con Uvicorn.
frontend	./frontend/Dockerfile	5173	5173	Servidor Vite para la interfaz React.

Variables de entorno principales

Archivo	Variable	Uso
.env.docker	DATABASE_URL	Cadena de conexión del backend hacia MySQL dentro de Docker.
.env.docker	JWT_SECRET_KEY	Clave usada para firmar tokens JWT; debe cambiarse antes de producción.
.env.docker	ACCESS_TOKEN_EXPIRE_MINUTES	Tiempo máximo de vida del token.
.env.docker	SESSION_INACTIVITY_MINUTES	Tiempo de inactividad antes de cerrar sesión.
.env.docker	SINGLE_ACTIVE_SESSION	Activa o desactiva la regla de una sola sesión por cuenta.
.env.docker	SMTP_ENABLED y SMTP_*	Configuración para envío de correos de recuperación.
frontend/.env	VITE_API_URL	URL del backend. Vacío permite usar automáticamente el mismo host con puerto 8000.
frontend/.env	VITE_SESSION_HEARTBEAT_MS	Intervalo de verificación de sesión desde el navegador.
frontend/.env	VITE_NOTIFICATION_POLL_MS	Intervalo de revisión de solicitudes pendientes para notificaciones.

Guía de despliegue con Docker

Requisitos previos

- Docker Desktop instalado y ejecutándose.
- Puerto 5173 libre para frontend.
- Puerto 8000 libre para backend.
- Puerto 3307 libre para MySQL desde el host.
- Tener el proyecto descomprimido y ubicarse en la carpeta donde está docker-compose.yml.

Paso 1: abrir terminal en la raíz del proyecto

```
cd "Sistema Inventario- Construccion"
```

Paso 2: revisar variables de entorno

- Verificar que exista el archivo .env.docker en la raíz del proyecto.

- Verificar que exista frontend/.env.
- En ambiente de presentación se puede dejar VITE_API_URL vacío para que el frontend use automáticamente el host actual y el puerto 8000.
- Antes de una entrega real, cambiar JWT_SECRET_KEY y no dejar credenciales sensibles visibles.

Paso 3: levantar los contenedores

```
docker compose up --build
```

Si se desea ejecutar en segundo plano: docker compose up -d --build

Paso 4: verificar que el sistema esté activo

Verificación	URL o comando	Resultado esperado
Frontend	http://localhost:5173	Debe cargar la pantalla de login.
Backend	http://localhost:8000/docs	Debe abrir la documentación automática de FastAPI.
Salud BD	http://localhost:8000/api/health/db	Debe responder un JSON indicando conexión correcta.
Logs	docker compose logs -f backend	No debe mostrar errores críticos de inicio.
Contenedores	docker compose ps	db, backend y frontend deben aparecer activos.

Paso 5: detener el sistema

```
docker compose down
```

Reiniciar base de datos desde cero

Los scripts 01_schema.sql y 02_seed_base.sql se ejecutan al crear por primera vez el volumen de MySQL. Si ya existe el volumen, los datos se conservan. Para borrar todo y volver a cargar los datos iniciales:

```
docker compose down -v
docker compose up --build
```

Advertencia: docker compose down -v elimina el volumen mysql_data y borra los datos guardados. Usarlo solo cuando se quiera reiniciar la base.

Guía de despliegue sin Docker

Este modo es útil para desarrollo, pero requiere instalar dependencias manualmente. Se recomienda Docker para una presentación porque reduce problemas de configuración entre computadoras.

Backend local

```
cd backend
python -m venv .env
# Windows:
.venv\Scripts\activate
# Linux/Mac:
# source .venv/bin/activate
pip install -r requirements.txt
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Frontend local

```
cd frontend
npm install
npm run dev -- --host 0.0.0.0
```

Base de datos local

- Instalar MySQL 8 o usar un servicio MySQL ya disponible.
- Crear la base de datos y usuario según la cadena DATABASE_URL configurada.
- Ejecutar database/01_schema.sql para crear tablas.
- Ejecutar database/02_seed_base.sql para cargar datos iniciales.
- Asegurar que el backend tenga DATABASE_URL y JWT_SECRET_KEY configurados.

Conexión desde varias laptops en red local

El proyecto ya está preparado para acceder desde otras laptops conectadas a la misma red. El frontend calcula automáticamente la URL del backend usando el mismo host donde se abrió la página y cambiando al puerto 8000.

Paso 1: levantar el sistema en la laptop host

```
docker compose up --build
```

Paso 2: obtener la IP de la laptop host

```
ipconfig
```

Buscar la Dirección IPv4, por ejemplo: 192.168.1.50.

Paso 3: abrir desde otra laptop

```
http://IP_DEL_HOST:5173
```

Ejemplo:
`http://192.168.1.50:5173`

Paso 4: probar backend desde otra laptop

```
http://IP_DEL_HOST:8000/api/health/db
```

Problemas comunes en red local

Problema	Causa probable	Solución
No abre el frontend desde otra laptop	Firewall o Vite no escucha en 0.0.0.0.	Permitir puerto 5173 y confirmar que npm run dev use --host 0.0.0.0.
Frontend abre, pero no conecta al backend	Puerto 8000 bloqueado o backend caído.	Probar /api/health/db y permitir puerto 8000 en firewall.
Funciona en host, no en otras laptops	Red configurada como Pública o aislamiento de clientes en router.	Cambiar red a Privada y desactivar AP/client isolation si existe.
La base no inicia	Puerto 3307 ocupado o volumen corrupto.	Liberar puerto, revisar logs o reiniciar volumen si no hay datos importantes.
Credenciales inválidas después de reiniciar	Se está usando un volumen antiguo con datos previos.	Usar las cuentas de ese volumen o recrear base con docker compose down -v.

Pruebas, verificación y solución de problemas

Pruebas unitarias

El proyecto incluye pruebas unitarias en backend/tests/unit/. Estas pruebas verifican reglas importantes sin depender de la base de datos real de MySQL.

```
docker compose up -d --build
docker compose exec backend pip install -r requirements-dev.txt
docker compose exec backend pytest
```

Comandos útiles de diagnóstico

Comando	Uso
docker compose ps	Ver estado de contenedores.
docker compose logs -f backend	Ver errores o actividad del backend.
docker compose logs -f frontend	Ver errores del frontend/Vite.
docker compose logs -f db	Ver estado de MySQL y carga de scripts SQL.
docker compose exec backend sh	Entrar al contenedor backend.
docker compose exec db mysql -u inventario -p sistematype	Entrar a MySQL desde el contenedor usando usuario configurado.
docker compose down	Detener servicios conservando datos.
docker compose down -v	Detener y borrar volumen de base de datos.

Checklist antes de exponer o entregar

- El login carga correctamente en <http://localhost:5173>.
- El backend responde en <http://localhost:8000/api/health/db>.
- Los cuatro roles pueden iniciar sesión con sus cuentas de prueba.
- Cada rol solo ve su menú permitido.
- El Supervisor de Almacén Central puede revisar solicitudes y despachar.
- El Supervisor de Sucursal puede solicitar reposición y recibir despachos.
- El Vendedor puede registrar ventas y el stock se descuenta.
- Los movimientos se registran después de compras, reposiciones y ventas.
- Las pruebas unitarias se ejecutan sin errores.
- En red local, otra laptop puede abrir http://IP_DEL_HOST:5173.

Recomendaciones para entrega o producción

Área	Recomendación
Seguridad	Cambiar JWT_SECRET_KEY, no subir credenciales reales, usar contraseñas fuertes y revisar permisos por rol.
SMTP	Usar contraseña de aplicación o servicio transaccional; revisar SPF/DKIM/DMARC para reducir correos en spam.
Base de datos	Hacer copias de seguridad periódicas y no exponer MySQL públicamente.
Frontend	Para producción, generar build con npm run build y servir archivos estáticos con Nginx u otro servidor.
Backend	Ejecutar sin --reload, usar APP_DEBUG=false y configurar logs.
HTTPS	Usar certificado SSL/TLS si se publica fuera de red local.
Variables	Separar ambientes: desarrollo, pruebas y producción.
Usuarios	Mantener una cuenta por persona y evitar compartir credenciales.
Pruebas	Agregar pruebas de integración para flujos completos: compra, reposición, venta y reportes.
Documentación	Mantener actualizados README, guía de instalación, roles y casos de uso.

Conclusión

El sistema presenta una arquitectura clara y suficiente para una aplicación web de inventario multirol. La separación entre frontend, backend y base de datos permite explicar fácilmente el flujo de información. La división por roles ayuda al trabajo en equipo y el despliegue con Docker Compose simplifica la ejecución en una laptop host o en una red local para pruebas y exposición.